



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Introduction

When I got my first game console in 1979—a way-cool Intellivision system by Mattel—the term “game engine” did not exist. Back then, video and arcade games were considered by most adults to be nothing more than toys, and the software that made them tick was highly specialized to both the game in question and the hardware on which it ran. Today, games are a multi-billion-dollar mainstream industry rivaling Hollywood in size and popularity. And the software that drives these now-ubiquitous three-dimensional worlds—*game engines* like Epic Games’ Unreal Engine 4, Valve’s Source engine and, Crytek’s CRYENGINE® 3, Electronic Arts DICE’s Frostbite™ engine, and the Unity game engine—have become fully featured reusable software development kits that can be licensed and used to build almost any game imaginable.

While game engines vary widely in the details of their architecture and implementation, recognizable coarse-grained patterns have emerged across both publicly licensed game engines and their proprietary in-house counterparts. Virtually all game engines contain a familiar set of core components, including the rendering engine, the collision and physics engine, the animation system, the audio system, the game world object model, the artificial intelligence system and so on. Within each of these components, a relatively small number of semi-standard design alternatives are also beginning to emerge.

There are a great many books that cover individual game engine subsystems, such as three-dimensional graphics, in exhaustive detail. Other books cobble together valuable tips and tricks across a wide variety of game technology areas. However, I have been unable to find a book that provides its reader with a reasonably complete picture of the entire gamut of components that make up a modern game engine. The goal of this book, then, is to take the reader on a guided hands-on tour of the vast and complex landscape of game engine architecture.

In this book you will learn:

- how real industrial-strength production game engines are architected;
- how game development teams are organized and work in the real world;
- which major subsystems and design patterns appear again and again in virtually every game engine;
- the typical requirements for each major subsystem;
- which subsystems are genre- or game-agnostic, and which ones are typically designed explicitly for a specific genre or game; and
- where the engine normally ends and the game begins.

We'll also get a first-hand glimpse into the inner workings of some popular game engines, such as Quake, Unreal and Unity, and some well-known middleware packages, such as the Havok Physics library, the OGRE rendering engine and Rad Game Tools' Granny 3D animation and geometry management toolkit. And we'll explore a number of proprietary game engines that I've had the pleasure to work with, including the engine Naughty Dog developed for its *Uncharted* and *The Last of Us* game series.

Before we get started, we'll review some techniques and tools for large-scale software engineering in a game engine context, including:

- the difference between logical and physical software architecture;
- configuration management, revision control and build systems; and
- some tips and tricks for dealing with one of the common development environments for C and C++, Microsoft Visual Studio.

In this book I assume that you have a solid understanding of C++ (the language of choice among most modern game developers) and that you understand basic software engineering principles. I also assume you have some

exposure to linear algebra, three-dimensional vector and matrix math and trigonometry (although we'll review the core concepts in Chapter 5). Ideally, you should have some prior exposure to the basic concepts of real time and event-driven programming. But never fear—I will review these topics briefly, and I'll also point you in the right direction if you feel you need to hone your skills further before we embark.

1.1 Structure of a Typical Game Team

Before we delve into the structure of a typical game engine, let's first take a brief look at the structure of a typical game development team. Game studios are usually composed of five basic disciplines: engineers, artists, game designers, producers and other management and support staff (marketing, legal, information technology/technical support, administrative, etc.). Each discipline can be divided into various subdisciplines. We'll take a brief look at each below.

1.1.1 Engineers

The engineers design and implement the software that makes the game, and the tools, work. Engineers are often categorized into two basic groups: *runtime* programmers (who work on the engine and the game itself) and *tools* programmers (who work on the offline tools that allow the rest of the development team to work effectively). On both sides of the runtime/tools line, engineers have various specialties. Some engineers focus their careers on a single engine system, such as rendering, artificial intelligence, audio or collision and physics. Some focus on gameplay programming and scripting, while others prefer to work at the systems level and not get too involved in how the game actually plays. Some engineers are generalists—jacks of all trades who can jump around and tackle whatever problems might arise during development.

Senior engineers are sometimes asked to take on a technical leadership role. Lead engineers usually still design and write code, but they also help to manage the team's schedule, make decisions regarding the overall technical direction of the project, and sometimes also directly manage people from a human resources perspective.

Some companies also have one or more technical directors (TD), whose job it is to oversee one or more projects from a high level, ensuring that the teams are aware of potential technical challenges, upcoming industry developments, new technologies and so on. The highest engineering-related position at a game studio is the chief technical officer (CTO), if the studio has one. The

CTO's job is to serve as a sort of technical director for the entire studio, as well as serving a key executive role in the company.

1.1.2 Artists

As we say in the game industry, "Content is king." The artists produce all of the visual and audio content in the game, and the quality of their work can literally make or break a game. Artists come in all sorts of flavors:

- *Concept artists* produce sketches and paintings that provide the team with a vision of what the final game will look like. They start their work early in the concept phase of development, but usually continue to provide visual direction throughout a project's life cycle. It is common for screenshots taken from a shipping game to bear an uncanny resemblance to the concept art.
- *3D modelers* produce the three-dimensional geometry for everything in the virtual game world. This discipline is typically divided into two sub-disciplines: foreground modelers and background modelers. The former create objects, characters, vehicles, weapons and the other objects that populate the game world, while the latter build the world's static background geometry (terrain, buildings, bridges, etc.).
- *Texture artists* create the two-dimensional images known as textures, which are applied to the surfaces of 3D models in order to provide detail and realism.
- *Lighting artists* lay out all of the light sources in the game world, both static and dynamic, and work with color, intensity and light direction to maximize the artfulness and emotional impact of each scene.
- *Animators* imbue the characters and objects in the game with motion. The animators serve quite literally as actors in a game production, just as they do in a CG film production. However, a game animator must have a unique set of skills in order to produce animations that mesh seamlessly with the technological underpinnings of the game engine.
- *Motion capture actors* are often used to provide a rough set of motion data, which are then cleaned up and tweaked by the animators before being integrated into the game.
- *Sound designers* work closely with the engineers in order to produce and mix the sound effects and music in the game.

- *Voice actors* provide the voices of the characters in many games.
- Many games have one or more *composers*, who compose an original score for the game.

As with engineers, senior artists are often called upon to be team leaders. Some game teams have one or more *art directors*—very senior artists who manage the look of the entire game and ensure consistency across the work of all team members.

1.1.3 Game Designers

The game designers' job is to design the interactive portion of the player's experience, typically known as *gameplay*. Different kinds of designers work at different levels of detail. Some (usually senior) game designers work at the macro level, determining the story arc, the overall sequence of chapters or levels, and the high-level goals and objectives of the player. Other designers work on individual levels or geographical areas within the virtual game world, laying out the static background geometry, determining where and when enemies will emerge, placing supplies like weapons and health packs, designing puzzle elements and so on. Still other designers operate at a highly technical level, working closely with gameplay engineers and/or writing code (often in a high-level scripting language). Some game designers are ex-engineers, who decided they wanted to play a more active role in determining how the game will play.

Some game teams employ one or more *writers*. A game writer's job can range from collaborating with the senior game designers to construct the story arc of the entire game, to writing individual lines of dialogue.

As with other disciplines, some senior designers play management roles. Many game teams have a game director, whose job it is to oversee all aspects of a game's design, help manage schedules, and ensure that the work of individual designers is consistent across the entire product. Senior designers also sometimes evolve into producers.

1.1.4 Producers

The role of producer is defined differently by different studios. In some game companies, the producer's job is to manage the schedule and serve as a human resources manager. In other companies, producers serve in a senior game design capacity. Still other studios ask their producers to serve as liaisons between the development team and the business unit of the company (finance, legal, marketing, etc.). Some smaller studios don't have producers at all. For

example, at Naughty Dog, literally everyone in the company, including the two co-presidents, plays a direct role in constructing the game; team management and business duties are shared between the senior members of the studio.

1.1.5 Other Staff

The team of people who directly construct the game is typically supported by a crucial team of support staff. This includes the studio's executive management team, the marketing department (or a team that liaises with an external marketing group), administrative staff and the IT department, whose job is to purchase, install and configure hardware and software for the team and to provide technical support.

1.1.6 Publishers and Studios

The marketing, manufacture and distribution of a game title are usually handled by a *publisher*, not by the game studio itself. A publisher is typically a large corporation, like Electronic Arts, THQ, Vivendi, Sony, Nintendo, etc. Many game studios are not affiliated with a particular publisher. They sell each game that they produce to whichever publisher strikes the best deal with them. Other studios work exclusively with a single publisher, either via a long-term publishing contract or as a fully owned subsidiary of the publishing company. For example, THQ's game studios are independently managed, but they are owned and ultimately controlled by THQ. Electronic Arts takes this relationship one step further, by directly managing its studios. *First-party developers* are game studios owned directly by the console manufacturers (Sony, Nintendo and Microsoft). For example, Naughty Dog is a first-party Sony developer. These studios produce games exclusively for the gaming hardware manufactured by their parent company.

1.2 What Is a Game?

We probably all have a pretty good intuitive notion of what a game is. The general term "game" encompasses board games like chess and *Monopoly*, card games like poker and blackjack, casino games like roulette and slot machines, military war games, computer games, various kinds of play among children, and the list goes on. In academia we sometimes speak of *game theory*, in which multiple agents select strategies and tactics in order to maximize their gains within the framework of a well-defined set of game rules. When used in the context of console or computer-based entertainment, the word "game"

usually conjures images of a three-dimensional virtual world featuring a humanoid, animal or vehicle as the main character under player control. (Or for the old geezers among us, perhaps it brings to mind images of two-dimensional classics like *Pong*, *Pac-Man*, or *Donkey Kong*.) In his excellent book, *A Theory of Fun for Game Design*, Raph Koster defines a game to be an interactive experience that provides the player with an increasingly challenging sequence of patterns which he or she learns and eventually masters [30]. Koster's assertion is that the activities of learning and mastering are at the heart of what we call "fun," just as a joke becomes funny at the moment we "get it" by recognizing the pattern.

For the purposes of this book, we'll focus on the subset of games that comprise two- and three-dimensional virtual worlds with a small number of players (between one and 16 or thereabouts). Much of what we'll learn can also be applied to HTML5/JavaScript games on the Internet, pure puzzle games like *Tetris*, or massively multiplayer online games (MMOG). But our primary focus will be on game engines capable of producing first-person shooters, third-person action/platform games, racing games, fighting games and the like.

1.2.1 Video Games as Soft Real-Time Simulations

Most two- and three-dimensional video games are examples of what computer scientists would call *soft real-time interactive agent-based computer simulations*. Let's break this phrase down in order to better understand what it means.

In most video games, some subset of the real world—or an imaginary world—is *modeled* mathematically so that it can be manipulated by a computer. The model is an approximation to and a simplification of reality (even if it's an *imaginary* reality), because it is clearly impractical to include every detail down to the level of atoms or quarks. Hence, the mathematical model is a *simulation* of the real or imagined game world. Approximation and simplification are two of the game developer's most powerful tools. When used skillfully, even a greatly simplified model can sometimes be almost indistinguishable from reality—and a lot more fun.

An *agent-based* simulation is one in which a number of distinct entities known as "agents" interact. This fits the description of most three-dimensional computer games very well, where the agents are vehicles, characters, fireballs, power dots and so on. Given the agent-based nature of most games, it should come as no surprise that most games nowadays are implemented in an object-oriented, or at least loosely object-based, programming language.

All interactive video games are *temporal simulations*, meaning that the virtual game world model is *dynamic*—the state of the game world changes over time as the game’s events and story unfold. A video game must also respond to unpredictable inputs from its human player(s)—thus *interactive temporal simulations*. Finally, most video games present their stories and respond to player input in real time, making them *interactive real-time simulations*. One notable exception is in the category of turn-based games like computerized chess or turn-based strategy games. But even these types of games usually provide the user with some form of real-time graphical user interface. So for the purposes of this book, we’ll assume that all video games have at least *some* real-time constraints.

At the core of every real-time system is the concept of a *deadline*. An obvious example in video games is the requirement that the screen be updated at least 24 times per second in order to provide the illusion of motion. (Most games render the screen at 30 or 60 frames per second because these are multiples of an NTSC monitor’s refresh rate.) Of course, there are many other kinds of deadlines in video games as well. A physics simulation may need to be updated 120 times per second in order to remain stable. A character’s artificial intelligence system may need to “think” at least once every second to prevent the appearance of stupidity. The audio library may need to be called at least once every 1/60 second in order to keep the audio buffers filled and prevent audible glitches.

A “soft” real-time system is one in which missed deadlines are not catastrophic. Hence, all video games are *soft real-time systems*—if the frame rate dies, the human player generally doesn’t! Contrast this with a *hard real-time system*, in which a missed deadline could mean severe injury to or even the death of a human operator. The avionics system in a helicopter or the control-rod system in a nuclear power plant are examples of hard real-time systems.

Mathematical models can be *analytic* or *numerical*. For example, the analytic (closed-form) mathematical model of a rigid body falling under the influence of constant acceleration due to gravity is typically written as follows:

$$y(t) = \frac{1}{2}gt^2 + v_0t + y_0. \quad (1.1)$$

An analytic model can be evaluated for any value of its independent variables, such as the time t in the above equation, given only the initial conditions v_0 and y_0 and the constant g . Such models are very convenient when they can be found. However, many problems in mathematics have no closed-form solution. And in video games, where the user’s input is unpredictable, we cannot hope to model the entire game analytically.

A numerical model of the same rigid body under gravity can be expressed

as follows:

$$y(t + \Delta t) = F(y(t), \dot{y}(t), \ddot{y}(t), \dots). \quad (1.2)$$

That is, the height of the rigid body at some future time $(t + \Delta t)$ can be found as a function of the height and its first, second, and possibly higher-order time derivatives at the current time t . Numerical simulations are typically implemented by running calculations repeatedly, in order to determine the state of the system at each discrete time step. Games work in the same way. A main “game loop” runs repeatedly, and during each iteration of the loop, various game systems such as artificial intelligence, game logic, physics simulations and so on are given a chance to calculate or update their state for the next discrete time step. The results are then “rendered” by displaying graphics, emitting sound and possibly producing other outputs such as force-feedback on the joystick.

1.3 What Is a Game Engine?

The term “game engine” arose in the mid-1990s in reference to first-person shooter (FPS) games like the insanely popular *Doom* by id Software. *Doom* was architected with a reasonably well-defined separation between its core software components (such as the three-dimensional graphics rendering system, the collision detection system or the audio system) and the art assets, game worlds and rules of play that comprised the player’s gaming experience. The value of this separation became evident as developers began licensing games and retooling them into new products by creating new art, world layouts, weapons, characters, vehicles and game rules with only minimal changes to the “engine” software. This marked the birth of the “mod community”—a group of individual gamers and small independent studios that built new games by modifying existing games, using free toolkits provided by the original developers.

Towards the end of the 1990s, some games like *Quake III Arena* and *Unreal* were designed with reuse and “modding” in mind. Engines were made highly customizable via scripting languages like id’s Quake C, and engine licensing began to be a viable secondary revenue stream for the developers who created them. Today, game developers can license a game engine and reuse significant portions of its key software components in order to build games. While this practice still involves considerable investment in custom software engineering, it can be much more economical than developing all of the core engine components in-house.

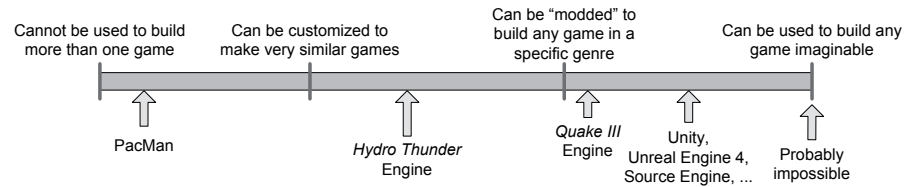


Figure 1.1. Game engine reusability gamut.

The line between a game and its engine is often blurry. Some engines make a reasonably clear distinction, while others make almost no attempt to separate the two. In one game, the rendering code might “know” specifically how to draw an orc. In another game, the rendering engine might provide general-purpose material and shading facilities, and “orc-ness” might be defined entirely in data. No studio makes a perfectly clear separation between the game and the engine, which is understandable considering that the definitions of these two components often shift as the game’s design solidifies.

Arguably a *data-driven architecture* is what differentiates a game engine from a piece of software that is a game but not an engine. When a game contains hard-coded logic or game rules, or employs special-case code to render specific types of game objects, it becomes difficult or impossible to reuse that software to make a different game. We should probably reserve the term “game engine” for software that is extensible and can be used as the foundation for many different games without major modification.

Clearly this is not a black-and-white distinction. We can think of a gamut of reusability onto which every engine falls. Figure 1.1 takes a stab at the locations of some well-known games/engines along this gamut.

One would think that a game engine could be something akin to Apple QuickTime or Microsoft Windows Media Player—a general-purpose piece of software capable of playing virtually *any* game content imaginable. However, this ideal has not yet been achieved (and may never be). Most game engines are carefully crafted and fine-tuned to run a particular game on a particular hardware platform. And even the most general-purpose multiplatform engines are really only suitable for building games in one particular genre, such as first-person shooters or racing games. It’s safe to say that the more general-purpose a game engine or middleware component is, the less optimal it is for running a particular game on a particular platform.

This phenomenon occurs because designing any efficient piece of software invariably entails making trade-offs, and those trade-offs are based on assumptions about how the software will be used and/or about the target

hardware on which it will run. For example, a rendering engine that was designed to handle intimate indoor environments probably won't be very good at rendering vast outdoor environments. The indoor engine might use a binary space partitioning (BSP) tree or portal system to ensure that no geometry is drawn that is being occluded by walls or objects that are closer to the camera. The outdoor engine, on the other hand, might use a less-exact occlusion mechanism, or none at all, but it probably makes aggressive use of level-of-detail (LOD) techniques to ensure that distant objects are rendered with a minimum number of triangles, while using high-resolution triangle meshes for geometry that is close to the camera.

The advent of ever-faster computer hardware and specialized graphics cards, along with ever-more-efficient rendering algorithms and data structures, is beginning to soften the differences between the graphics engines of different genres. It is now possible to use a first-person shooter engine to build a strategy game, for example. However, the trade-off between generality and optimality still exists. A game can always be made more impressive by fine-tuning the engine to the specific requirements and constraints of a particular game and/or hardware platform.

1.4 Engine Differences across Genres

Game engines are typically somewhat genre specific. An engine designed for a two-person fighting game in a boxing ring will be very different from a massively multiplayer online game (MMOG) engine or a first-person shooter (FPS) engine or a real-time strategy (RTS) engine. However, there is also a great deal of overlap—all 3D games, regardless of genre, require some form of low-level user input from the joypad, keyboard and/or mouse, some form of 3D mesh rendering, some form of heads-up display (HUD) including text rendering in a variety of fonts, a powerful audio system, and the list goes on. So while the Unreal Engine, for example, was designed for first-person shooter games, it has been used successfully to construct games in a number of other genres as well, including the wildly popular third-person shooter franchise *Gears of War* by Epic Games, the hit action-adventure games in the *Batman: Arkham* series by Rocksteady Studios, the well-known fighting game *Tekken 7* by Bandai Namco Studios, and the first three role-playing third-person shooter games in the *Mass Effect* series by BioWare.

Let's take a look at some of the most common game genres and explore some examples of the technology requirements particular to each.



Figure 1.2. *Overwatch* by Blizzard Entertainment (Xbox One, PlayStation 4, Windows). (See Color Plate I.)

1.4.1 First-Person Shooters (FPS)

The first-person shooter (FPS) genre is typified by games like *Quake*, *Unreal Tournament*, *Half-Life*, *Battlefield*, *Destiny*, *Titanfall* and *Overwatch* (see Figure 1.2). These games have historically involved relatively slow on-foot roaming of a potentially large but primarily corridor-based world. However, modern first-person shooters can take place in a wide variety of virtual environments including vast open outdoor areas and confined indoor areas. Modern FPS traversal mechanics can include on-foot locomotion, rail-confined or free-roaming ground vehicles, hovercraft, boats and aircraft. For an overview of this genre, see http://en.wikipedia.org/wiki/First-person_shooter.

First-person games are typically some of the most technologically challenging to build, probably rivaled in complexity only by third-person shooters, action-platformer games, and massively multiplayer games. This is because first-person shooters aim to provide their players with the illusion of being immersed in a detailed, hyperrealistic world. It is not surprising that many of the game industry's big technological innovations arose out of the games in this genre.

First-person shooters typically focus on technologies such as:

- efficient rendering of large 3D virtual worlds;

- a responsive camera control/aiming mechanic;
- high-fidelity animations of the player's virtual arms and weapons;
- a wide range of powerful handheld weaponry;
- a forgiving player character motion and collision model, which often gives these games a "floaty" feel;
- high-fidelity animations and artificial intelligence for the non-player characters (NPCs)—the player's enemies and allies; and
- small-scale online multiplayer capabilities (typically supporting between 10 and 100 simultaneous players), and the ubiquitous "death match" gameplay mode.

The rendering technology employed by first-person shooters is almost always highly optimized and carefully tuned to the particular type of environment being rendered. For example, indoor "dungeon crawl" games often employ binary space partitioning trees or portal-based rendering systems. Outdoor FPS games use other kinds of rendering optimizations such as occlusion culling, or an offline sectorization of the game world with manual or automated specification of which target sectors are visible from each source sector.

Of course, immersing a player in a hyperrealistic game world requires much more than just optimized high-quality graphics technology. The character animations, audio and music, rigid body physics, in-game cinematics and myriad other technologies must all be cutting-edge in a first-person shooter. So this genre has some of the most stringent and broad technology requirements in the industry.

1.4.2 Platformers and Other Third-Person Games

"Platformer" is the term applied to third-person character-based action games where jumping from platform to platform is the primary gameplay mechanic. Typical games from the 2D era include *Space Panic*, *Donkey Kong*, *Pitfall!* and *Super Mario Brothers*. The 3D era includes platformers like *Super Mario 64*, *Crash Bandicoot*, *Rayman 2*, *Sonic the Hedgehog*, the *Jak and Daxter* series (Figure 1.3), the *Ratchet & Clank* series and *Super Mario Galaxy*. See <http://en.wikipedia.org/wiki/Platformer> for an in-depth discussion of this genre.

In terms of their technological requirements, platformers can usually be lumped together with third-person shooters and third-person action/adventure games like *Just Cause 2*, *Gears of War 4* (Figure 1.4), the *Uncharted* series, the *Resident Evil* series, the *The Last of Us* series, *Red Dead Redemption 2*, and the list goes on.

Third-person character-based games have a lot in common with first-person shooters, but a great deal more emphasis is placed on the main character's abilities and locomotion modes. In addition, high-fidelity full-body character animations are required for the player's avatar, as opposed to the somewhat less-taxing animation requirements of the "floating arms" in a typical FPS game. It's important to note here that almost all first-person shooters have an online multiplayer component, so a full-body player avatar must be rendered in addition to the first-person arms. However, the fidelity of these FPS player avatars is usually not comparable to the fidelity of the non-player characters in these same games; nor can it be compared to the fidelity of the player avatar in a third-person game.

In a platformer, the main character is often cartoon-like and not particularly realistic or high-resolution. However, third-person shooters often feature a highly realistic humanoid player character. In both cases, the player character typically has a very rich set of actions and animations.

Some of the technologies specifically focused on by games in this genre include:



Figure 1.3. *Jak II* by Naughty Dog (Jak, Dexter, Jak and Dexter, and Jak II © 2003, 2013/™ SIE. Created and developed by Naughty Dog, PlayStation 2.) (See Color Plate II.)



Figure 1.4. *Gears of War 4* by The Coalition (Xbox One). (See Color Plate III.)

- moving platforms, ladders, ropes, trellises and other interesting locomotion modes;
- puzzle-like environmental elements;
- a third-person “follow camera” which stays focused on the player character and whose rotation is typically controlled by the human player via the right joystick (on a console) or the mouse (on a PC—note that while there are a number of popular third-person shooters on a PC, the platformer genre exists almost exclusively on consoles); and
- a complex camera collision system for ensuring that the view point never “clips” through background geometry or dynamic foreground objects.

1.4.3 Fighting Games

Fighting games are typically two-player games involving humanoid characters pummeling each other in a ring of some sort. The genre is typified by games like *Soul Calibur* and *Tekken 3* (see Figure 1.5). The Wikipedia page http://en.wikipedia.org/wiki/Fighting_game provides an overview of this genre.

Traditionally games in the fighting genre have focused their technology efforts on:

- a rich set of fighting animations;
- accurate hit detection;
- a user input system capable of detecting complex button and joystick combinations; and
- crowds, but otherwise relatively static backgrounds.

Since the 3D world in these games is small and the camera is centered on the action at all times, historically these games have had little or no need for world subdivision or occlusion culling. They would likewise not be expected to employ advanced three-dimensional audio propagation models, for example.

Modern fighting games like EA's *Fight Night Round 4* and NetherRealm Studios' *Injustice 2* (Figure 1.6) have upped the technological ante with features like:

- high-definition character graphics;
- realistic skin shaders with subsurface scattering and sweat effects;
- photo-realistic lighting and particle effects;
- high-fidelity character animations; and



Figure 1.5. *Tekken 3* by Namco (PlayStation). (See Color Plate IV.)

- physics-based cloth and hair simulations for the characters.

It's important to note that some fighting games like Ninja Theory's *Heavenly Sword* and *For Honor* by Ubisoft Montreal take place in a large-scale virtual world, not a confined arena. In fact, many people consider this to be a separate genre, sometimes called a *brawler*. This kind of fighting game can have technical requirements more akin to those of a third-person shooter or a strategy game.

1.4.4 Racing Games

The racing genre encompasses all games whose primary task is driving a car or other vehicle on some kind of track. The genre has many subcategories. Simulation-focused racing games ("sims") aim to provide a driving experience that is as realistic as possible (e.g., *Gran Turismo*). Arcade racers favor over-the-top fun over realism (e.g., *San Francisco Rush*, *Cruis'n USA*, *Hydro Thunder*). One subgenre explores the subculture of street racing with tricked out consumer vehicles (e.g., *Need for Speed*, *Juiced*). Kart racing is a subcategory in which popular characters from platformer games or cartoon characters from TV are re-cast as the drivers of whacky vehicles (e.g., *Mario Kart*, *Jak X*, *Freaky Flyers*). Racing games need not always involve time-based competition. Some kart racing games, for example, offer



Figure 1.6. *Injustice 2* by NetherRealm Studios (PlayStation 4, Xbox One, Android, iOS, Microsoft Windows). (See Color Plate V.)



Figure 1.7. *Gran Turismo Sport* by Polyphony Digital (PlayStation 4). (See Color Plate VI.)

modes in which players shoot at one another, collect loot or engage in a variety of other timed and untimed tasks. For a discussion of this genre, see http://en.wikipedia.org/wiki/Racing_game.

A racing game is often very linear, much like older FPS games. However, travel speed is generally much faster than in an FPS. Therefore, more focus is placed on very long corridor-based tracks, or looped tracks, sometimes with various alternate routes and secret short-cuts. Racing games usually focus all their graphic detail on the vehicles, track and immediate surroundings. As an example of this, Figure 1.7 shows a screenshot from the latest installment in the well-known *Gran Turismo* racing game series, *Gran Turismo Sport*, developed by Polyphony Digital and published by Sony Interactive Entertainment. However, kart racers also devote significant rendering and animation bandwidth to the characters driving the vehicles.

Some of the technological properties of a typical racing game include the following techniques:

- Various “tricks” are used when rendering distant background elements, such as employing two-dimensional cards for trees, hills and mountains.
- The track is often broken down into relatively simple two-dimensional regions called “sectors.” These data structures are used to optimize rendering and visibility determination, to aid in artificial intelligence and path finding for non-human-controlled vehicles, and to solve many other technical problems.
- The camera typically follows behind the vehicle for a third-person per-



Figure 1.8. *Age of Empires* by Ensemble Studios (Windows). (See Color Plate VII.)

spective, or is sometimes situated inside the cockpit first-person style.

- When the track involves tunnels and other “tight” spaces, a good deal of effort is often put into ensuring that the camera does not collide with background geometry.

1.4.5 Strategy Games

The modern strategy game genre was arguably defined by *Dune II: The Building of a Dynasty* (1992). Other games in this genre include *Warcraft*, *Command & Conquer*, *Age of Empires* and *Starcraft*. In this genre, the player deploys the battle units in his or her arsenal strategically across a large playing field in an attempt to overwhelm his or her opponent. The game world is typically displayed at an oblique top-down viewing angle. A distinction is often made between turn-based strategy games and real-time strategy (RTS). For a discussion of this genre, see https://en.wikipedia.org/wiki/Strategy_video_game.

The strategy game player is usually prevented from significantly changing the viewing angle in order to see across large distances. This restriction permits developers to employ various optimizations in the rendering engine of a strategy game.



Figure 1.9. *Total War: Warhammer 2* by Creative Assembly (Windows). (See Color Plate VIII.)

Older games in the genre employed a grid-based (cell-based) world construction, and an orthographic projection was used to greatly simplify the renderer. For example, Figure 1.8 shows a screenshot from the classic strategy game *Age of Empires*.

Modern strategy games sometimes use perspective projection and a true 3D world, but they may still employ a grid layout system to ensure that units and background elements, such as buildings, align with one another properly. A popular example, *Total War: Warhammer 2*, is shown in Figure 1.9.

Some other common practices in strategy games include the following techniques:

- Each unit is relatively low-res, so that the game can support large numbers of them on-screen at once.
- Height-field terrain is usually the canvas upon which the game is designed and played.
- The player is often allowed to build new structures on the terrain in addition to deploying his or her forces.
- User interaction is typically via single-click and area-based selection of units, plus menus or toolbars containing commands, equipment, unit types, building types, etc.



Figure 1.10. *World of Warcraft* by Blizzard Entertainment (Windows, MacOS). (See Color Plate IX.)

1.4.6 Massively Multiplayer Online Games (MMOG)

The massively multiplayer online game (MMOG or just MMO) genre is typified by games like *Guild Wars 2* (AreaNet/NCsoft), *EverQuest* (989 Studios/SOE), *World of Warcraft* (Blizzard) and *Star Wars Galaxies* (SOE/Lucas Arts), to name a few. An MMO is defined as any game that supports huge numbers of simultaneous players (from thousands to hundreds of thousands), usually all playing in one very large, *persistent* virtual world (i.e., a world whose internal state persists for very long periods of time, far beyond that of any one player's gameplay session). Otherwise, the gameplay experience of an MMO is often similar to that of their small-scale multiplayer counterparts. Subcategories of this genre include MMO role-playing games (MMORPG), MMO real-time strategy games (MMORTS) and MMO first-person shooters (MMOFPS). For a discussion of this genre, see <http://en.wikipedia.org/wiki/MMOG>. Figure 1.10 shows a screenshot from the hugely popular MMORPG *World of Warcraft*.

At the heart of all MMOGs is a very powerful battery of servers. These servers maintain the authoritative state of the game world, manage users signing in and out of the game, provide inter-user chat or voice-over-IP (VoIP) services and more. Almost all MMOGs require users to pay some kind of regular

subscription fee in order to play, and they may offer micro-transactions within the game world or out-of-game as well. Hence, perhaps the most important role of the central server is to handle the billing and micro-transactions which serve as the game developer's primary source of revenue.

Graphics fidelity in an MMO is almost always lower than its non-massively multiplayer counterparts, as a result of the huge world sizes and extremely large numbers of users supported by these kinds of games.

Figure 1.11 shows a screen from Bungie's latest FPS game, *Destiny 2*. This game has been called an MMOFPS because it incorporates some aspects of the MMO genre. However, Bungie prefers to call it a "shared world" game because unlike a traditional MMO, in which a player can see and interact with literally any other player on a particular server, *Destiny* provides "on-the-fly match-making." This permits the player to interact only with the other players with whom they have been matched by the server; this matchmaking system has been significantly improved for *Destiny 2*. Also unlike a traditional MMO, the graphics fidelity in *Destiny 2* is on par with first- and third-person shooters.

We should note here that the game *Player Unknown's Battlegrounds* (PUBG) has recently popularized a subgenre known as *battle royale*. This type of game blurs the line between regular multiplayer shooters and massively multiplayer online games, because they typically pit on the order of 100 players against each other in an online world, employing a survival-based "last man standing" gameplay style.



Figure 1.11. *Destiny 2* by Bungie, © 2018 Bungie Inc. (Xbox One, PlayStation 4, PC) (See Color Plate X.)

1.4.7 Player-Authored Content

As social media takes off, games are becoming more and more collaborative in nature. A recent trend in game design is toward *player-authored content*. For example, Media Molecule's *LittleBigPlanet*,TM *LittleBigPlanet*TM 2 (Figure 1.12) and *LittleBigPlanet*TM 3: *The Journey Home* are technically *puzzle platformers*, but their most notable and unique feature is that they encourage players to create, publish and share their own game worlds. Media Molecule's latest installment in this engaging genre is *Dreams* for the PlayStation 4 (Figure 1.13).

Perhaps the most popular game today in the player-created content genre is *Minecraft* (Figure 1.14). The brilliance of this game lies in its simplicity: *Minecraft* game worlds are constructed from simple cubic voxel-like elements mapped with low-resolution textures to mimic various materials. Blocks can be solid, or they can contain items such as torches, anvils, signs, fences and panes of glass. The game world is populated with one or more player characters, animals such as chickens and pigs, and various “mobs”—good guys like villagers and bad guys like zombies and the ubiquitous *creepers* who sneak up on unsuspecting players and explode (only scant moments after warning the player with the “hiss” of a burning fuse).

Players can create a randomized world in *Minecraft* and then dig into the generated terrain to create tunnels and caverns. They can also construct their own structures, ranging from simple terrain and foliage to vast and complex



Figure 1.12. *LittleBigPlanet*TM 2 by Media Molecule, © 2014 Sony Interactive Entertainment (PlayStation 3). (See Color Plate XI.)



Figure 1.13. *Dreams* by Media Molecule, © 2017 Sony Computer Computer Europe (PlayStation 4). (See Color Plate XII.)

buildings and machinery. Perhaps the biggest stroke of genius in *Minecraft* is *redstone*. This material serves as “wiring,” allowing players to lay down circuitry that controls pistons, hoppers, mine carts and other dynamic elements in the game. As a result, players can create virtually anything they can imagine, and then share their worlds with their friends by hosting a server and inviting them to play online.



Figure 1.14. *Minecraft* by Markus “Notch” Persson / Mojang AB (Windows, MacOS, Xbox 360, PlayStation 3, PlayStation Vita, iOS). (See Color Plate XIII.)

1.4.8 Virtual, Augmented and Mixed Reality

Virtual, augmented and mixed reality are exciting new technologies that aim to immerse the viewer in a 3D world that is either entirely generated by a computer, or is augmented by computer-generated imagery. These technologies have many applications outside the game industry, but they have also become viable platforms for a wide range of gaming content.

1.4.8.1 Virtual Reality

Virtual reality (VR) can be defined as an immersive multimedia or computer-simulated reality that simulates the user's presence in an environment that is either a place in the real world or in an imaginary world. Computer-generated VR (CG VR) is a subset of this technology in which the virtual world is exclusively generated via computer graphics. The user views this virtual environment by donning a headset such as HTC Vive, Oculus Rift, Sony PlayStation VR, Samsung Gear VR or Google Daydream View. The headset displays the content directly in front of the user's eyes; the system also tracks the movement of the headset in the real world, so that the virtual camera's movements can be perfectly matched to those of the person wearing the headset. The user typically holds devices in his or her hands which allow the system to track the movements of each hand. This allows the user to interact in the virtual world: Objects can be pushed, picked up or thrown, for example.

1.4.8.2 Augmented and Mixed Reality

The terms *augmented reality* (AR) and *mixed reality* (MR) are often confused or used interchangeably. Both technologies present the user with a view of the real world, but with computer graphics used to enhance the experience. In both technologies, a viewing device like a smart phone, tablet or tech-enhanced pair of glasses displays a real-time or static view of a real-world scene, and computer graphics are overlaid on top of this image. In real-time AR and MR systems, accelerometers in the viewing device permit the virtual camera's movements to track the movements of the device, producing the illusion that the device is simply a window through which we are viewing the actual world, and hence giving the overlaid computer graphics a strong sense of realism.

Some people make a distinction between these two technologies by using the term "augmented reality" to describe technologies in which computer graphics are overlaid on a live, direct or indirect view of the real world, but are not anchored to it. The term "mixed reality," on the other hand, is more often

applied to the use of computer graphics to render imaginary objects which are anchored to the real world and appear to exist within it. However, this distinction is by no means universally accepted.

Here are a few examples of AR technology in action:

- The U.S. Army provides its soldiers with improved tactical awareness using a system dubbed “tactical augmented reality” (TAR)—it overlays a video-game-like heads-up display (HUD) complete with a mini-map and object markers onto the soldier’s view of the real world (<https://youtu.be/x8p19j8C6VI>).
- In 2015, Disney demonstrated some cool AR technology that renders a 3D cartoon character on top of a sheet of real-world paper on which a 2D version of the character is colored with a crayon (<https://youtu.be/SWzurBQ81CM>).
- PepsiCo also pranked commuting Londoners with an AR-enabled bus stop. People sitting in the bus stop enclosure were treated to AR images of a prowling tiger, a meteor crashing, and an alien tentacle grabbing unwitting passers by off the street (<https://youtu.be/Go9rf9GmYpM>).

And here are a few examples of MR:

- Starting with Android 8.1, the camera app on the Pixel 1 and Pixel 2 supports *AR Stickers*, a fun feature that allows users to place animated 3D objects and characters into videos and photos.
- Microsoft’s HoloLens is another example of mixed reality. It overlays world-anchored graphics onto a live video image, and can be used for a wide range of applications including education and training, engineering, health care, and entertainment.

1.4.8.3 VR/AR/MR Games

The game industry is currently experimenting with VR and AR/MR technologies, and is trying to find its footing within these new media. Some traditional 3D games have been “ported” to VR, yielding very interesting, if not particularly innovative, experiences. But perhaps more exciting, entirely new game genres are starting to emerge, offering gameplay experiences that could not be achieved without VR or AR/MR.

For example, *Job Simulator* by Owlchemy Labs plunges the user into a virtual job museum run by robots, and asks them to perform tongue-in-cheek approximations of various real-world jobs, making use of game mechanics

that simply wouldn't work on a non-VR platform. Owlchemy's next installment, *Vacation Simulator*, applies the same whimsical sense of humour and art style to a world in which the robots of *Job Simulator* invite the player to relax and perform various tasks. Figure 1.15 shows a screenshot from another innovative (and somewhat disturbing!) game for HTC Vive called *Accounting*, from the creators of "Rick & Morty" and *The Stanley Parable*.

1.4.8.4 VR Game Engines

VR game engines are technologically similar in many respects to first-person shooter engines, and in fact many FPS-capable engines such as Unity and Unreal Engine support VR "out of the box." However, VR games differ from FPS games in a number of significant ways:

- *Stereoscopic rendering.* A VR game needs to render the scene twice, once for each eye. This doubles the number of graphics primitives that must be rendered, although other aspects of the graphics pipeline such as visibility culling can be performed only once per frame, since the eyes are reasonably close together. As such, a VR game isn't quite as expensive to render as the same game would be to render in split-screen multiplayer mode, but the principle of rendering each frame twice from two (slightly) different virtual cameras is the same.
- *Very high frame rate.* Studies have shown that VR running at below 90



Figure 1.15. *Accounting* by Squanchtendo and Crows Crows Crows (HTC Vive). (See Color Plate XIV.)

frames per second is likely to induce disorientation, nausea, and other negative user effects. This means that not only do VR systems need to render the scene twice per frame, they need to do so at 90+ FPS. This is why VR games and applications are generally required to run on high-powered CPU and GPU hardware.

- *Navigation issues.* In an FPS game, the player can simply walk around the game world with the joypad or the WASD keys. In a VR game, a small amount of movement can be realized by the user physically walking around in the real world, but the safe physical play area is typically quite small (the size of a small bathroom or closet). Travelling by “flying” tends to induce nausea as well, so most games opt for a point-and-click teleportation mechanism to move the virtual player/camera across larger distances. Various real-world devices have also been conceived that allow a VR user to “walk” in place with their feet in order to move around in a VR world.

Of course, VR makes up for these limitations somewhat by enabling new user interaction paradigms that aren’t possible in traditional video games. For example,

- users can reach in the real world to touch, pick up and throw objects in the virtual world;
- a player can dodge an attack in the virtual world by dodging physically in the real world;
- new user interface opportunities are possible, such as having floating menus attached to one’s virtual hands, or seeing a game’s credits written on a whiteboard in the virtual world;
- a player can even pick up a pair of virtual VR goggles and place them onto his or her head, thereby transporting them into a “nested” VR world—an effect that might best be called “VR-ception.”

1.4.8.5 Location-Based Entertainment

Games like *Pokémon Go* neither overlay graphics onto an image of the real world, nor do they generate a completely immersive virtual world. However, the user’s view of the computer-generated world of *Pokémon Go* does react to movements of the user’s phone or tablet, much like a 360-degree video. And the game is aware of your actual location in the real world, prompting you to go searching for Pokémon in nearby parks, malls and restaurants. This kind of game can’t really be called AR/MR, but neither does it fall into the VR category. Such a game might be better described as a form of *location-based*

entertainment, although some people do use the AR moniker for these kinds of games.

1.4.9 Other Genres

There are of course many other game genres which we won't cover in depth here. Some examples include:

- sports, with subgenres for each major sport (football, baseball, soccer, golf, etc.);
- role-playing games (RPG);
- God games, like *Populous* and *Black & White*;
- environmental/social simulation games, like *SimCity* or *The Sims*;
- puzzle games like *Tetris*;
- conversions of non-electronic games, like chess, card games, go, etc.;
- web-based games, such as those offered at Electronic Arts' Pogo site;

and the list goes on.

We have seen that each game genre has its own particular technological requirements. This explains why game engines have traditionally differed quite a bit from genre to genre. However, there is also a great deal of technological overlap between genres, especially within the context of a single hardware platform. With the advent of more and more powerful hardware, differences between genres that arose because of optimization concerns are beginning to evaporate. It is therefore becoming increasingly possible to reuse the same engine technology across disparate genres, and even across disparate hardware platforms.

1.5 Game Engine Survey

1.5.1 The Quake Family of Engines

The first 3D first-person shooter (FPS) game is generally accepted to be *Castle Wolfenstein 3D* (1992). Written by id Software of Texas for the PC platform, this game led the game industry in a new and exciting direction. id Software went on to create *Doom*, *Quake*, *Quake II* and *Quake III*. All of these engines are very similar in architecture, and I will refer to them as the Quake family of engines. Quake technology has been used to create many other games and even other engines. For example, the lineage of *Medal of Honor* for the PC platform goes something like this: